PRACTICAL GUIDE (RANKED RECOMMENDATION)

SEPTEMBER 2026

Open-Source Paths into LLM Inference for SRE and Go Engineers

A ranked guide to the Kubernetes-native inference projects where Go, Kubernetes and operations skills carry over, with project health data, first contributions and a 30/90/180-day roadmap (September 2026)

PUBLISHED
September 2026

EVIDENCE BASE
126 cited sources

PREPARED BY
Boku research agents

Research question: "Opensource Projects to contribute as a Someone with SRE, DevOps, Golang Experience pivoting in Inference, AI and LLM world with no idea". Compiled from public sources by an automated multi-agent research pipeline. Claims are cited to their sources; numbered citations refer to the companion file open-source-paths-into-llm-inference-sre-go-engineers.references.json. Verify material facts before relying on them.

CONTENTS

In this report

— The short answer

— Key findings

01 **Where to spend your contribution time**

02 **How the inference stack fits together, and who owns what since 2026**

03 **Project health and newcomer-friendliness compared**

04 **Project profiles and concrete first contributions**

05 **What carries over, and what you must learn**

06 **Career signal: what employers ask for**

07 **Staged roadmap: first 30, 90 and 180 days**

08 **Limitations and data gaps**

— What to do next

A Methodology and evidence quality

THE SHORT ANSWER

Open-Source Paths into LLM Inference for SRE and Go Engineers

Start with the Kubernetes layer of the LLM inference stack, not the serving engines. By share of repository bytes, the routing, control-plane and scheduling projects are 64.3% to 98.5% Go. vLLM, TensorRT-LLM and llama.cpp contain 0% Go, and SGLang 0.1%[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21]. vLLM itself is 85.0% Python[4,8,16,17,18]. In order, the recommended projects are: **(1) llm-d**, **(2) KServe**, **(3) Kueue and LeaderWorkerSet**, **(4) NVIDIA KAI Scheduler**, and **(5) vLLM, but only at its edges**: metrics, tracing, benchmarks and CI.

llm-d is the best single place to start. It was founded in May 2025 by Red Hat, Google Cloud, IBM Research, CoreWeave and NVIDIA, and the CNCF accepted it as a Sandbox project on 2026-03-12[22,23,24,25,26,27,28]. After Gateway API Inference Extension (GAIE) v1.6.0 in August 2026, llm-d owns the full-featured inference router, the Endpoint Picker (EPP)[29,30,31,32,33]. Its router, KV-cache libraries, GPU-free vLLM simulator, batch gateway, autoscaler and async processor are all Go repositories[34]. Its SIGs for Installation, Observability, Benchmarking and Autoscaling correspond directly to SRE work[35].

KServe comes second. It is a CNCF incubating project (accepted 2025-09-29), 64.3% Go, and shipped four minor releases, v0.18 to v0.21, between April and September 2026[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,36,37,38]. Kueue (94.3% Go) and LeaderWorkerSet (95.9% Go) are the easiest places to land Go pull requests under Kubernetes-SIG governance[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,39,40]. On 2026-09-30 Kueue had 4 open good-first-issues and 7 help-wanted issues[41,42,43,44,45,46,47,48,49]. KAI Scheduler (98.5% Go) published 10 tagged releases between mid-August and 2026-09-28, but all 10 of its maintainers work at NVIDIA[50,51,52]. vLLM has about 93.0k stars, and 315 contributors worked on v0.30.0 alone. That makes it the place to learn inference, but it offers little Go work[26,32,52,53,54,55,56,57,58,59,60,61].

The main problem is getting in. Labelled beginner work is scarce exactly where the inference signal is strongest. On 2026-09-30, the llm-d main repository had 1 good-first-issue, GAIE had 0, and KServe had 1 good-first-issue and 0 help-wanted issues[41,42,43,44,45,46,47,48,49]. None of the core inference projects ran their own LFX Mentorship or CNCF GSoC 2026 projects[62,63,64,65]. In practice you get in through SIG meetings, Slack channels and maintainers' explicit requests. Activity data is also partial: monthly active contributors and merged-PR counts for the past 12 months were not available for any project.

Demand is real, but evidence about specific skills is thin. In the CNCF 2025 survey, 66% of organisations hosting generative AI models used Kubernetes for some or all of their inference[66,67]. Linux Foundation research projects a net +60% hiring effect for AI-specific roles in 2026[68]. In a sample of just 3 postings from 2026, all 3 require Kubernetes and name Go[69,70,71]. The Red Hat posting accepts SRE backgrounds and prefers open-source contributions to KServe or vLLM[69]. The practical path runs in stages: Helm, CI and observability work that needs only latency and KV-cache vocabulary, then router and autoscaler logic, and engine internals last.

NOT VERIFIED BY FACT-CHECK

The fact checker could not verify the following. Treat any statement in this report that touches on them as unconfirmed.

- No verified data available for AI chip market size in 2025
- No verified data available for NVIDIA AI chip revenue in 2025
- No verified data available for AMD AI chip market share in 2025
- No verified growth rate data for AI chip market in 2025

KEY FINDINGS

What the evidence shows

01 **Go stops at the serving engine: the control-plane and scheduling projects are 64.3–98.5% Go, while vLLM is 0% Go.**

KAI Scheduler is 98.5% Go, LWS 95.9%, Kueue 94.3%, GAIE 88.6% and KServe 64.3%. vLLM is 85.0% Python, and Dynamo's core is 52.8% Rust[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21].

02 **llm-d is the best single project for a Go SRE, with at least 6 Go repositories and SIGs for installation, observability, benchmarking and autoscaling.**

Its CONTRIBUTING file asks for load and chaos testing, docs and dependency updates. The CNCF asks for production deployment patterns and open benchmarking standards[27,34,35,72].

03 **In 2026, routing logic moved from the Kubernetes SIG repository to llm-d.**

WG Serving concluded (announced 2026-02-26), and GAIE v1.6.0 (August 2026) moved the full EPP and latency predictor into the llm-d org. GAIE now covers only the InferencePool API, conformance and a lightweight EPP[29,30,31,32,33,73].

04 **Labelled newcomer work is scarce: 1 good-first-issue in llm-d's main repo, 0 in GAIE and 1 in KServe on 2026-09-30.**

vLLM (10) and Kueue (4) had the most. All 3 good-first-issues in llm-d-router require prefix-cache/TTFT or KV-transfer knowledge[33,41,42,43,44,45,46,74].

05 **Observability is the shortest bridge from SRE to inference work.**

vLLM exports Prometheus metrics for TTFT, ITL, queue depth and KV-cache usage. llm-d renamed its router metrics to llm_d_epp_* in v0.10, and its August 2026 tracing post asks for missing spans and Collector recipes[75,76,77,78].

06 **All 3 sampled 2026 inference postings require Kubernetes and name Go, but 3 postings is a small sample.**

vLLM appears in 2 of the 3, and llm-d and KServe in 1 each. The Red Hat posting accepts SRE backgrounds and prefers KServe/vLLM contributions[69,70,71].

01 SECTION

Where to spend your contribution time

The ranking weighs four things: how much Go the project contains, how closely its open work matches SRE tasks, how active and well governed it is, and how directly employers name it. Every project ranked in the top four is mostly Go at the component level. The only one outside that group is vLLM, which ranks high on career signal and learning value but has no Go code[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21].

TABLE 1

Ranked candidate projects for a Go-fluent SRE, September 2026

RANK / PROJECT	GO SHARE (% OF REPO BYTES)	STACK LAYER	GOVERNANCE AND BACKERS	GITHUB STARS	OPEN GOOD-FIRST-ISSUES
1. llm-d	Router predecessor 96.8%; router, KV-cache, sim, autoscaler repos are Go[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,34]	Routing, KV-aware scheduling, autoscaling	CNCF Sandbox (2026-03-12); Red Hat, Google, IBM maintainers; NVIDIA, CoreWeave founders[22,23,24,25,26,27,28,79]	4.7k	1 (main repo); 3 in llm-d-router[33,74]
2. KServe	64.3%	K8s control plane (LLMInferenceService)	CNCF Incubating (2025-09-29); steering from Bloomberg, IBM, Red Hat, Google, NVIDIA and others[22,23,24,25,26,80,81]	6.1k	1
3a. Kueue	94.3%	Batch and quota scheduling	Kubernetes SIG Scheduling[39,40]	3.0k	4
3b. LeaderWorkerSet	95.9%	Multi-node and P/D replicas	Kubernetes SIG Apps[32,73]	846	Not documented
4. KAI Scheduler	98.5%	GPU scheduling	CNCF Sandbox (2025-12-21); all 10 maintainers NVIDIA[22,23,24,25,26,51,52]	1.5k	Not documented
5. vLLM (edges only)	0% (Python 85.0%)[4,8,16,17,18]	Serving engine	PyTorch Foundation-hosted[54,82]	93.0k	10
Secondary: GAIE	88.6%	InferencePool API, conformance	Kubernetes SIG Network[32,73]	775	0
Secondary: Ollama	69.2%	Local model runner	Not documented	181.9k	3

Source: [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [32], [33], [34], [39], [40], [41], [42], [43], [44], [45], [46], [51], [52], [53], [54], [55], [56], [57], [58], [73], [74], [79], [80], [81], [82]. Data as of 2026-09-30. Go share from GitHub language byte counts; stars and issue counts are point-in-time on 2026-09-30[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,26,32,41,42,43,44,45,46,52,53,54,55,56,57,58].

- **llm-d**: the routing, KV-cache and autoscaling work is in Go, and SIGs exist for installation, observability and benchmarking[34,35].
- **KServe**: its LLMInferenceService controller turns LLM deployments into Gateway, HTTPRoute, InferencePool and LWS resources, which is standard controller and Helm work[83,84].
- **Kueue + LWS**: the easiest Go entry points, run by Kubernetes-SIGs; LWS supplies the multi-node and P/D replicas that llm-d and KServe depend on[27,41,42,43,44,45,46,56,83,84].
- **KAI Scheduler**: fast release cadence and a direct NVIDIA hiring signal, but a single-vendor maintainer group[50,51,52,71].
- **vLLM**: learn from it and contribute to its metrics, tracing, benchmarks and CI; do not expect Go work there[75,85,86,87,88,89].

> **LABELS WILL NOT FIND YOU THE WORK**
>
> The inference control-plane projects have almost no labelled beginner issues: 1 good-first-issue in llm-d's main repo and 0 in GAIE on 2026-09-30[41,42,43,44,45,46]. Plan to find work through SIG meetings, Slack channels and maintainers' posted requests, not through issue labels.

The ranking follows the employer evidence that exists. Kubernetes-based inference is common: 66% of organisations hosting generative AI models use Kubernetes for some or all inference[66,67]. Postings for inference roles name Kubernetes, Go, vLLM, llm-d, KServe and KAI Scheduler[69,70,71]. This suggests that contributing to the Go control plane while learning the engine through its metrics matches what hiring teams list.

02 SECTION

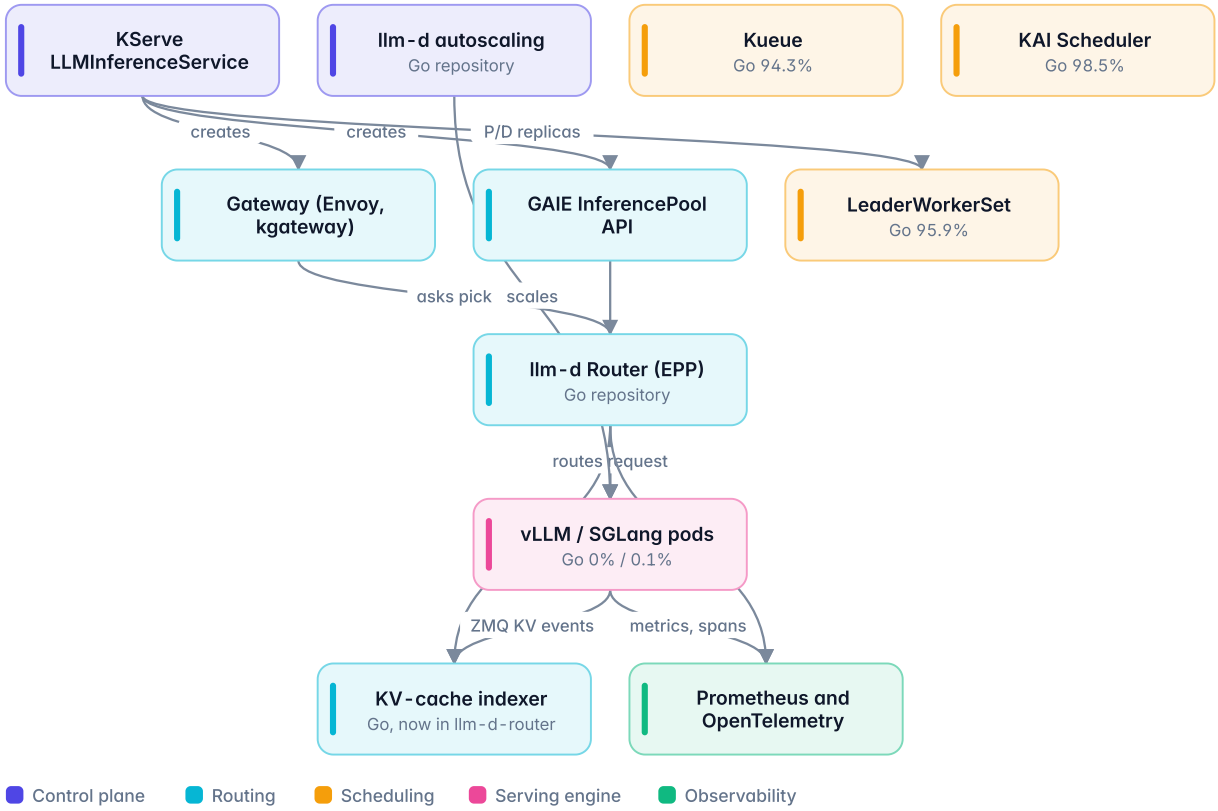
How the inference stack fits together, and who owns what since 2026

An inference request follows a short, documented path. In llm-d, a client request reaches the llm-d Router. The Router is an L7 proxy plus the Endpoint Picker, which scores model-server pods using real-time metrics and chooses one from an InferencePool of vLLM or SGLang pods[90]. KV-cache management, prefill/decode (P/D) disaggregation, latency-prediction routing, batch handling and the autoscaler are optional add-ons to this core[90].

KServe creates the same structure from one custom resource. Its LLMInferenceService controller creates a Gateway, an HTTPRoute, an InferencePool, an EPP scheduler pod and, for P/D, a LeaderWorkerSet. The KV cache itself moves between prefill and decode pods inside vLLM over RDMA[83,84]. The KV-cache indexer is a Go library. It consumes block create and evict events that vLLM publishes over ZMQ and returns a prefix-hit score for each pod[91].

FIGURE 1

LLM inference stack on Kubernetes, with Go share by component



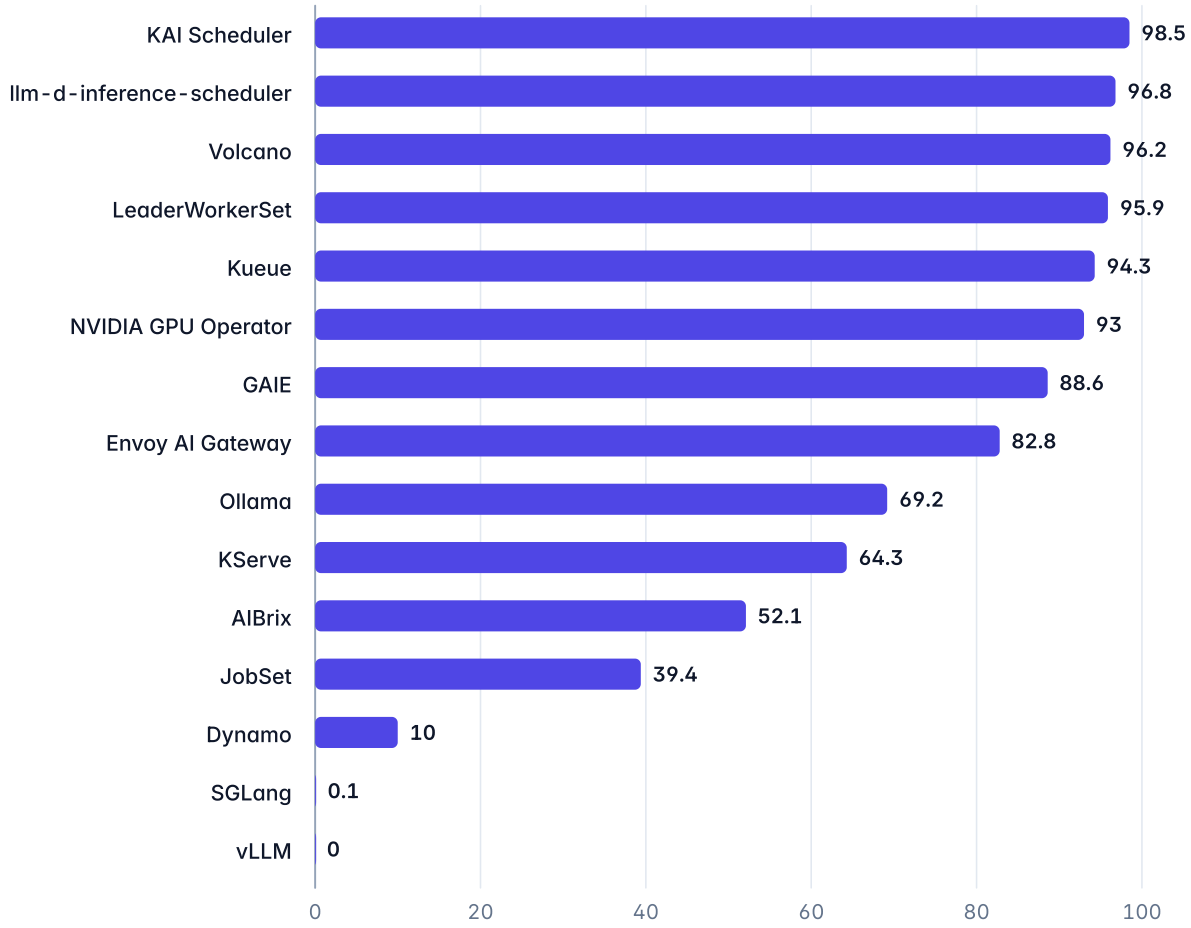
Source: [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [34], [76], [77], [78], [83], [84], [90], [91].

The chart below shows where Go work actually exists. Of the 21 repositories measured, the Kubernetes control-plane, scheduling and ops projects sit at 82.8–98.5% Go. KServe (64.3%) and AIBrix (52.1%) mix Go with Python, and the serving engines have essentially none[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21]. NVIDIA Dynamo is the exception to watch. Its core is 52.8% Rust, but it ships a Go Kubernetes operator, Grove[4,8,16,17,18,92,93].

FIGURE 2

Go share of main repository, selected inference projects

% of repository bytes



Source: [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21]. Data as of 2026-09-30. Derived from GitHub languages API byte counts fetched 2026-09-30. llm-d-inference-scheduler is the predecessor of llm-d-router; the router's own share was not measured.

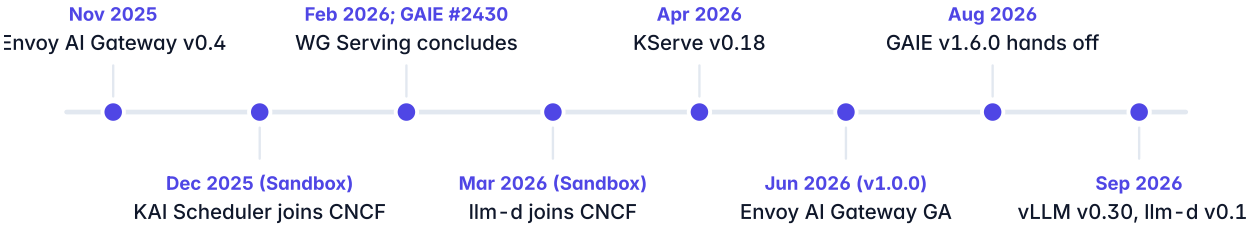
2026 moved the router into llm-d

Kubernetes WG Serving concluded in 2026 (CNCF announcement 2026-02-26). GAIE moved to SIG Network, LWS to SIG Apps and Inference Perf to SIG Scalability. The CNCF named llm-d and AIBrix as better places to drive inference requirements into Kubernetes[32,73]. On the same day, GAIE issue #2430 proposed moving the Endpoint Picker and the InferenceObjective and InferenceModelRewrite APIs into llm-d. The reason given was that the work needs deep AI/ML expertise and close integration with model servers[31,32,33].

GAIE v1.6.0 (August 2026) completed the move. The full EPP, Body-Based Routing and the latency predictor left the repository, and GAIE now covers the InferencePool API, CRDs, conformance and a lightweight EPP[29,30]. Its community meetings now take place in the llm-d Router meeting[31,32,33]. In practice, routing-logic contributions belong in llm-d-router, and GAIE is the place for API and conformance work.

FIGURE 3

Key inference-infrastructure events, Nov 2025 to Sep 2026



Source: [22], [23], [24], [25], [26], [29], [30], [31], [32], [33], [36], [37], [38], [56], [59], [60], [61], [73], [94], [95].

Release activity over the year was steady across the stack. llm-d shipped v0.7.0 through v0.10.0 between May and September 2026. In v0.8.0 it renamed the inference scheduler to the router endpoint picker, and in v0.10.0 it moved to vLLM v0.30.0[56,94]. KServe shipped v0.18.0 to v0.21.0 (2026-04-29 to 2026-09-25)[36,37,38]. LWS shipped v0.9.0 to v0.11.0, adding DisaggregatedSet, per-role HPA and gang scheduling[96,97]. Envoy AI Gateway went from v0.4.0 (November 2025) to v1.0.0 GA (June 2026)[95].

03 SECTION

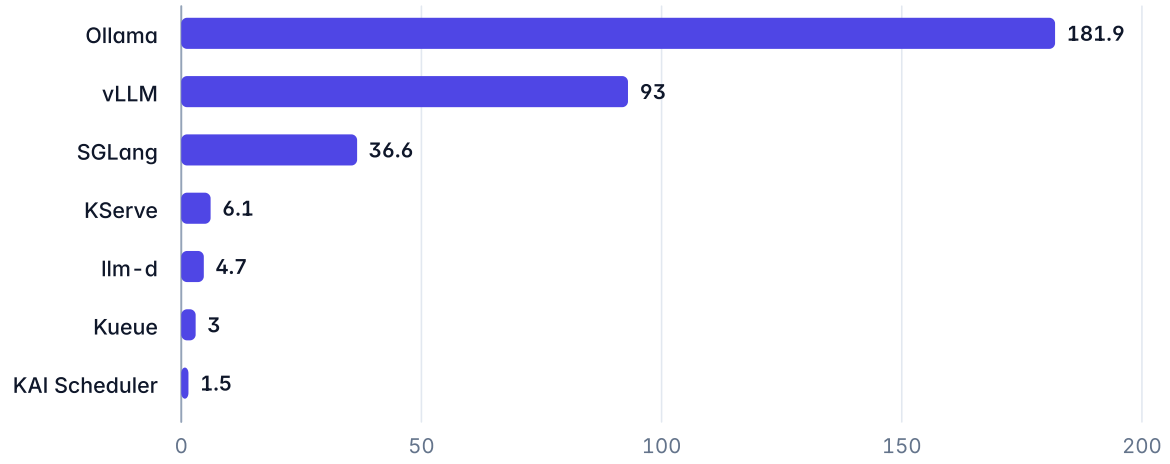
Project health and newcomer-friendliness compared

Stars show visibility, not how easy a project is to contribute to. Ollama (181.9k) and vLLM (93.0k) are far ahead of the Kubernetes-native projects, which range from 6.1k (KServe) down to 846 (LWS) and 775 (GAIE)[26,32,52,53,54,55,56,57,58]. For a contributor this cuts both ways: smaller projects have fewer reviewers but also fewer people competing for the same issues.

FIGURE 4

GitHub stars on 2026-09-30

thousand stars



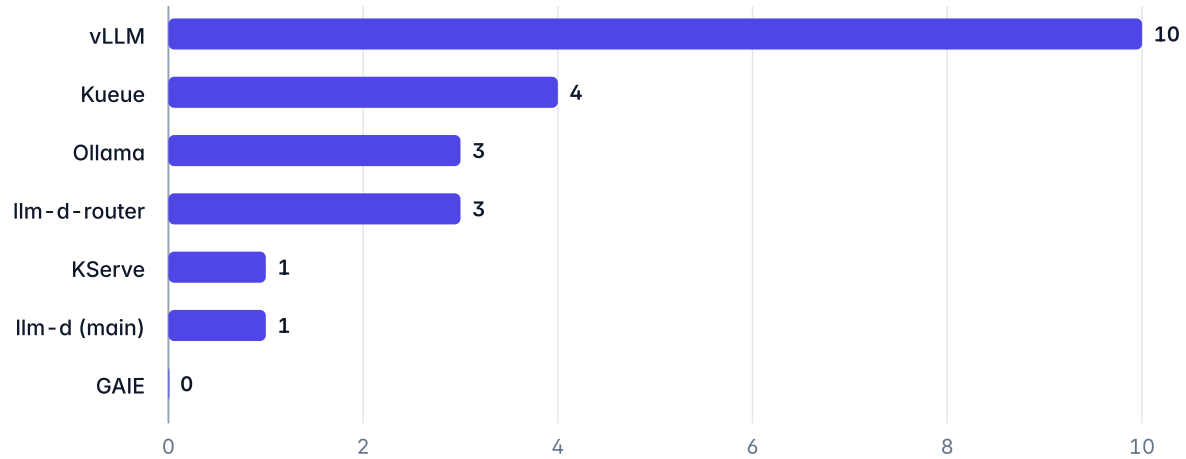
Source: [26], [32], [52], [53], [54], [55], [56], [57], [58]. Data as of 2026-09-30. Point-in-time GitHub counts. Not shown: LeaderWorkerSet 846 and GAIE 775 stars.

Backlog size shows the same split. vLLM had 2,500+ open issues and 5,000+ open PRs on 2026-09-30. Kueue had 516 and 285, KServe 73 and 127, llm-d 63 and 104, and GAIE only 16 and 5[26,32,52,53,54,55,56,57,58]. A large open-PR queue like vLLM's usually means slower review for first-time contributors. That is an inference, not something the projects report.

FIGURE 5

Open 'good first issue' counts by repository

open issues



Source: [33], [41], [42], [43], [44], [45], [46], [74]. Data as of 2026-09-30. GitHub label searches, 2026-09-30 (llm-d-router, September 2026). Not collected for LWS, KAI Scheduler or SGLang.

Labelled entry points are sparse everywhere, and the inference control plane has the fewest. Help-wanted counts were vLLM 11, Kueue 7 and KServe 0[47,48,49]. The llm-d-router good-first-issues are not beginner tasks in the usual sense. All 3 involve prefix-cache/TTFT

scoring or peer-to-peer KV transfer[33,74]. On labels alone, Kueue is the most accessible Go project[41,42,43,44,45,46,47,48,49].

TABLE 2

LFX Insights health indicators, past 365 days

PROJECT	HEALTH SCORE (/100)	MAINTAINERS WITH MERGE RIGHTS	MEDIAN RESPONSE TIME	CONTRIBUTING ORGANISATIONS
llm-d	86	135	4 days	760
vLLM	80	917	3 days	5,591
KServe	74	41	2 days	579

Source: [98], [99], [100]. Data as of 2025-10 to 2026-09 (past 365 days). The exact window is not stated beyond 'past 365 days'. vLLM's 917 'maintainers with merge rights' has an unclear definition and should not be compared across projects[98,99,100].

LLM-D IS NOT STAGNANT, DESPITE ONE FLAG

LFX Insights says llm-d's release cadence has slowed and gives it a development-activity sub-score of 15/25[56,94,98,99,100]. The GitHub releases page shows 5 releases from v0.7.0 to v0.10.0 between 2026-05-12 and 2026-09-29[56,94]. LFX probably undercounts because llm-d's code is split across many repositories and work moved in from GAIE during 2026[31,32,33,34]. That explanation is an inference.

No reliable 12-month trend exists for contributor activity. OSS Insight's cumulative PR-creator series shows vLLM rising from 2,243 to 3,082 between October 2025 and August 2026, and Kueue from 204 to 256. However, several months are missing and every repository jumps anomalously in September 2026[101,102,103,104]. OSS Insight labels its own data as degraded lower bounds. Its vLLM star count for September 2026 is 60,046, against about 93.0k on GitHub[54,105]. These series show direction only.

TABLE 3

Community channels, meetings and 2026 mentoring by project

PROJECT	CHAT	RECURRING MEETINGS	2026 MENTORING (LFX / GSOC)
llm-d	Slack, including #sig-router, #sig-observability, #sig-installation[35]	Bi-weekly Wed 12:30 PM ET standup; SIGs weekly or bi-weekly[26,32,35,52,53,56,57,58]	None run directly[62]
GAIE	#gateway-api-inference-extension (Kubernetes Slack)[26,32,52,53,56,57,58]	Merged into llm-d Router meeting[31,32,33]	None[62]
KServe	Slack[26,32,52,53,56,57,58]	Not documented	Kubeflow GSoC: KServe Models Web App[63,64,65]
Kueue	#wg-batch, wg-batch mailing list[39,40]	Not documented	Headlamp Kueue plugin (LFX)[62,106,107]
LeaderWorkerSet	#sig-apps (Kubernetes Slack)[26,32,52,53,56,57,58]	Not documented	Not documented
KAI Scheduler	#kai-scheduler (CNCF Slack)[26,32,52,53,56,57,58]	Bi-weekly Mon 17:00 CEST[26,32,52,53,56,57,58]	None[62]
vLLM	slack.vllm.ai, discuss.vllm.ai[54,108]	Bi-weekly office hours; weekly SIG meetings[54,108]	None run directly[106,107]

Source: [26], [31], [32], [33], [35], [39], [40], [52], [53], [54], [56], [57], [58], [62], [63], [64], [65], [106], [107], [108]. Data as of 2026-09.

Structured mentoring exists only in adjacent projects. 2026 LFX terms included a Volcano KV-cache-aware scheduler e2e test suite, HAMi GPU observability dashboards and a Headlamp Kueue plugin[62]. The only inference-related idea on CNCF's GSoC 2026 list was kgateway's benchmarking of inference routing extensions[63,64,65]. These are reasonable fallbacks for someone who wants a mentor more than a specific project.

04 SECTION

Project profiles and concrete first contributions

1. llm-d: the Go core of Kubernetes-native inference

llm-d has 7 listed maintainers: 3 from Red Hat, 2 from Google and 2 from IBM[79]. Its launch partners include AMD, Cisco, Hugging Face, Intel, Lambda and Mistral AI[27,28]. The Go repositories are the router (EPP), llm-d-kv-cache, llm-d-inference-sim, llm-d-batch-gateway, llm-d-autoscaling and llm-d-async. The benchmark harness and latency predictor are Python[34]. New components require a proposal in ./proposals/. Contributions use DCO sign-off and lazy consensus[27,72].

The best place to start is llm-d-inference-sim. It imitates vLLM's TTFT and inter-token latency, emits KV-cache ZMQ events, exposes some of vLLM's Prometheus metrics and can inject failures, all without a GPU[109]. That makes CI, e2e and resilience-testing work possible on a laptop cluster, and it matches CONTRIBUTING's request for load and chaos testing[27,72]. Observability is the second opening. The August 2026 tracing post asks for issues about missing spans and Collector recipes[78]. The v0.10 metric rename to llm_d_epp_* means dashboards and alerts need updating[76,77].

2. KServe: controllers and Helm with a clear career signal

KServe had 19 maintainers and 300+ contributors when it moved to incubation (announced 2025-11-11)[80,81]. Its releases in 2026 point to operations work: restricted Pod Security Standards by default in v0.18, JUnit E2E output and HPA/KEDA status in v0.19, and DRA resourceClaims and independent CRD installation in v0.21[36,37,38]. LFX Insights reports the fastest median maintainer response of the three projects it scores, at 2 days[98,99,100]. The weakness is direction: ROADMAP.md is still the 2024–2025 version, and there were 0 help-wanted issues on 2026-09-30[37,47,48,49,110].

3. Kueue and LeaderWorkerSet: the easiest Go pull requests

Kueue's open good-first-issues include ProvisioningRequest docs (#5172), kueue-viz configuration (#3880) and SLSA attestations (#1466)[41,42,43,44,45,46]. Its help-wanted list includes DisaggregatedSet integration and metrics in load testing[47,48,49]. Kueue runs e2e tests on Kubernetes 1.34–1.37 and had v0.20.0-rc.2 out on 2026-09-29[57,96,97]. LWS matters because llm-d and KServe use it for multi-node and P/D replicas. Its 2026 releases added DisaggregatedSet, per-role HPA and gang scheduling[27,56,83,84,96,97].

4. KAI Scheduler: for GPU scheduling roles

KAI Scheduler provides gang scheduling and hierarchical queues with DRF, and has been tested on clusters of more than 10,000 GPUs. A CNCF blog of 2026-07-23 says its roadmap is now community-directed[24,111]. Its v0.18.0 release added fractional GPU support, pod resize accounting and FIPS mode[50]. An NVIDIA scheduler posting names the project directly[71]. The trade-off is that all 10 maintainers are at NVIDIA[51,52].

5. vLLM: contribute at the edges, learn from the core

vLLM's metrics, OpenTelemetry tracing (off by default) and benchmarking CLI are the realistic places for an SRE to contribute[75,78,85,86,87,88,89]. It runs bi-weekly office hours and weekly SIG meetings[54,108]. Two secondary options are also worth knowing. The OpenTelemetry GenAI semantic conventions now live in their own repository, whose schema URL was still marked 'TODO' in September 2026[112,113]. Envoy AI Gateway, 82.8% Go, added OTel GenAI tracing in v1.1.0[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,95].

TABLE 4

Concrete first-contribution targets

REPOSITORY	TARGET	TYPE	ML LEVEL
llm-d/llm-d	#1525 guides: request draining (opened 2026-05-16)[41,42,43,44,45,46]	Docs	1
llm-d-inference-sim	CI/e2e and failure-injection scenarios[109]	Testing	1
llm-d-router / Helm	Missing spans, Collector recipes for a backend[78]	Observability	1
llm-d-autoscaling	Follow the OTel pattern from #1272 (now closed)[114]	Go observability	1

REPOSITORY	TARGET	TYPE	ML LEVEL
llm-d dashboards	Update alerts for the llm_d_epp_* rename[76,77]	Prometheus	1
kserve/kserve	#6260 (opened 2026-09-21); LLMInferenceService e2e[41,42,43,44,45,46,83,84]	Controller, testing	1–2
kubernetes-sigs/kueue	#5172 docs, #3880 kueue-viz config, #1466 SLSA[41,42,43,44,45,46]	Docs, build	1
kubernetes-sigs/kueue	DisaggregatedSet integration (help wanted)[47,48,49]	Go feature	2
llm-d-router	#655 load-aware scorers; #2650 TTFT load gate[33,74]	Go routing	2
vllm-project/vllm	Metrics, tracing, vllm bench[75,85,86,87,88,89]	Observability, benchmarks	1–2

Source: [33], [41], [42], [43], [44], [45], [46], [47], [48], [49], [74], [75], [76], [77], [78], [83], [84], [85], [86], [87], [88], [89], [91], [109], [114], [115]. Data as of 2026-09-30. ML levels follow the three-level concept primer in the next section[74,83,91,109,115].

05 SECTION

What carries over, and what you must learn

The skills overlap most on observability and install/test work. The matrix below links each SRE skill to documented contribution areas. Every project has work for Go controllers and operators except vLLM. Prometheus and OpenTelemetry work shows up in llm-d, vLLM and, through tracing on its roadmap, KServe[34,37,75,78,110].

TABLE 5

Skill-mapping matrix: SRE/DevOps/Go skills against project contribution areas

SKILL	LLM-D	KSERVE	KUEUE / LWS	KAI SCHEDULER	VLLM
Go controllers and operators	Router, autoscaler, batch gateway[34]	LLMInferenceService controller[83,84]	Core codebase, ~94–96% Go[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21]	Core codebase, 98.5% Go[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21]	— (0% Go)[1,2,3,4,5,6,7,8,9,10,11,12,
Helm / K8s install	SIG Installation: llm-d-infra, modelservice[35]	Independent CRD install, v0.21[36,37,38]	kueue-viz backend config[47,48,49]	Not documented	—
CI and e2e	inference-sim tests; load, chaos testing[27,72,109]	JUnit E2E output, v0.19[36,37,38]	e2e on K8s 1.34–1.37; SLSA[41,42,43,44,45,46,57]	Not documented	Not documented
Prometheus / Grafana	llm_d_epp_* dashboards[76,77]	Not documented	Metrics in load testing[47,48,49]	Not documented	vllm:* metrics[75]
OpenTelemetry	End-to-end spans; autoscaler tracing[78,114]	InferenceGraph tracing on roadmap[37,110]	—	—	--otlp-traces-endpoin
Autoscaling	SIG Autoscaling; llm-d-autoscaling[35]	LWS autoscaling; HPA/KEDA status[36,37,38]	LWS per-role HPA[96,97]	—	—
Load testing	llm-d-benchmark (helm, helmfile)[85,86,87,88,89]	—	Metrics in load testing[47,48,49]	—	vllm bench, GuideLLM[8

Source: [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [27], [34], [35], [36], [37], [38], [41], [42], [43], [44], [45], [46], [47], [48], [49], [57], [72], [75], [76], [77], [78], [83], [84], [85], [86], [87], [88], [89], [96], [97], [109], [110], [114]. Data as of 2026-09.

Benchmarking uses Python, not Go. inference-perf, GuideLLM, vllm bench and llm-d-benchmark are all Python, and NVIDIA's genai-perf now directs users to AIPerf[85,86,87,88,89]. llm-d-benchmark runs other harnesses on Kubernetes using helm, helmfile and kubectl. Contributions there are mainly orchestration work with some Python[85,86,87,88,89].

Practitioners already treat inference signals as SLIs. At KubeCon Europe 2026 (2026-03-26), a Red Hat Senior Principal SRE co-presented a talk that framed TTFT, cache hit ratio, routing latency and GPU utilisation as SLIs, instrumented with Prometheus, OpenTelemetry and Grafana[116]. This suggests that SLO and alerting experience counts as inference expertise, provided you know the vocabulary.

The ML knowledge you need can be learned in stages, not all at once. Level 1 work needs only TTFT, ITL/TPOT, throughput and goodput, KV-cache usage and queue depth. Level 2 adds prefix caching, block hashing and the difference between prefill and decode. Level 3, the engine and KV-transfer work, needs continuous batching, paged attention, chunked prefill, KV connectors, parallelism and quantization[74,83,91,109,115]. This staging is an interpretation based on each component's scope.

TABLE 6

Inference concept primer by contribution level

LEVEL	WORK IT UNLOCKS	KEY CONCEPTS	WHERE YOU SEE IT IN VLLM	FREE RESOURCE
1	Helm, CI on the simulator, dashboards, OTel plumbing	TTFT, ITL/TPOT, goodput, KV-cache usage, queue depth	vllm:time_to_first_token_seconds, vllm:kv_cache_usage_perc, vllm:num_requests_waiting[75]	vLLM metrics design doc; llm-d tracing post, 2026-08-26[75,78]
2	EPP scorers, KV-cache indexer, autoscaler	Prefix caching, block hashing, prefill vs decode	vllm:prefix_cache_hits, vllm:request_prefill_time_seconds[75]	llm-d posts on token-aware routing (2026-08-17) and P2P KV sharing (2026-08-15)[115,117,118]

LEVEL	WORK IT UNLOCKS	KEY CONCEPTS	WHERE YOU SEE IT IN VLLM	FREE RESOURCE
3	vLLM engine, P/D sidecar, KV transfer	Continuous batching, paged attention, chunked prefill, KV connectors, quantization	vllm:model_forward_time_milliseconds, tracing only[75,78]	'Inside vLLM' (2025-09-05); Hugging Face continuous batching (2025-11-25)[115,117,118]

Source: [74], [75], [78], [83], [91], [109], [115], [117], [118]. Data as of 2026-09.

The two engine explainers are older but still the clearest free introductions. The vLLM anatomy post of 2025-09-05 covers the scheduler, paged attention, continuous batching, chunked prefill and P/D disaggregation. The Hugging Face post of 2025-11-25 builds from attention to the KV cache to continuous batching[115,117,118]. Reading these alongside llm-d's 2026 blog posts covers every concept in the table[115,117,118].

06 SECTION

Career signal: what employers ask for

Kubernetes is now a common place to run inference. The CNCF 2025 Annual Cloud Native Survey (628 respondents, published January 2026) found that 66% of organisations hosting generative AI models use Kubernetes for some or all inference. Only 7% deploy AI models daily[66,67,119,120]. Kubernetes production use among container users reached 82% in 2025, up from 66% in 2023, but 44% of organisations do not yet run AI/ML workloads on it[66,121].

Hiring research points in the same direction. The Linux Foundation 2026 State of Tech Talent Report (400 respondents, fielded February 2026) projects a net +31% hiring effect from AI overall and +60% for AI-specific roles in 2026[68]. Organisations prefer upskilling (57%) to external hiring (49%)[68,122]. That favours engineers who can show new skills in public. Indeed Hiring Lab found that 37% of US software-development posting growth from May 2025 to May 2026 came from roles with AI in the title[123].

TABLE 7

Three sampled 2026 inference postings

EMPLOYER AND ROLE	POSTED	REQUIRED	PREFERRED / NAMED PROJECTS	SALARY (USD)
Red Hat, Forward Deployed Engineer, AI Inference	Removed 2026-06-22	Kubernetes CRDs/operators, Gateway API; Python and Go; vLLM, llm-d; KV caching, P/D, batching; SRE or infra background accepted	OSS contributions to KServe, vLLM or Kubernetes; Inference Gateway	184,940–342,490
CoreWeave, Senior Software Engineer, Inference	2026-09-22	Python or Go; production Kubernetes; Prometheus, Grafana, OpenTelemetry; P95/P99 tuning	vLLM, Triton, TensorRT-LLM, Ray Serve	152,000–204,000
NVIDIA, Senior Software Engineer, Kubernetes AI Scheduler	2026-06-25	Go; Kubernetes internals; CRD/controller design	KAI-Scheduler; Kubernetes SIGs/WGs	Not disclosed

Source: [69], [70], [71]. Data as of 2026-09. Collected from tier 3–4 job aggregators; a sample for illustration, not a market measure[69,70,71].

Across these 3 postings, Kubernetes and Go appear in all 3, vLLM in 2, and llm-d, KServe and GPU scheduling in 1 each[69,70,71]. The Red Hat posting is the most relevant for this reader. It explicitly accepts backend, SRE or infrastructure experience and lists open-source contributions to KServe or vLLM as preferred[69].

> THIN EVIDENCE ON TRANSITIONS
>
> No source reviewed for this report documents a named SRE or DevOps engineer moving into an inference role through open-source contribution. No project publishes typical times from first contribution to reviewer. Three postings show which skills appear in job ads, not how often they appear across the market[69,70,71].

07 SECTION

Staged roadmap: first 30, 90 and 180 days

Days 0–30: get running and get visible. Deploy llm-d with llm-d-inference-sim on a local cluster so you can run the full routing path without a GPU[109]. Join #sig-installation, #sig-observability and the bi-weekly Wednesday standup[26,32,35,52,53,56,57,58]. Land one docs or build fix, such as llm-d #1525 or a Kueue good-first-issue like #5172 or #1466[41,42,43,44,45,46]. Learn the Level 1 vocabulary from vLLM's metric names[75].

Days 31–90: own an operational area. Take on observability: update dashboards and alerts for the llm_d_epp_* rename, and file or fix missing spans and Collector recipes as the tracing post asks[76,77,78]. The autoscaler tracing request in #1272 is a model for this kind of Go work[114]. Run llm-d-benchmark with inference-perf or GuideLLM to learn how load shapes change TTFT and ITL[85,86,87,88,89]. In parallel, pick up KServe LLMInferenceService e2e or controller work, or the #6260 good-first-issue[41,42,43,44,45,46,83,84].

Days 91–180: move into inference logic. Work on a router scorer or autoscaler feature. llm-d-router #655 (load-aware scorers) and #2650 (TTFT load gate) are good Level 2 targets[33,74]. Larger changes go through a proposal in ./proposals/[27,72]. Read the vLLM scheduler code with the anatomy post alongside to build Level 3 understanding[115,117,118]. If a mentor would help, LFX terms and GSoC run on fixed windows. GSoC 2026 applications ran from 2026-03-16 to 03-31[63,64,65].

TABLE 8

30/90/180-day contribution and learning milestones

WINDOW	PROJECT	CONTRIBUTION	CONCEPTS TO LEARN	EVIDENCE OF PROGRESS
Days 0–30	llm-d + inference-sim	Local deploy; docs fix (#1525)[41,42,43,44,45,46,109]	TTFT, ITL, queue depth[74,83,91,109,115]	First merged PR; attend SIG meeting
Days 0–30	Kueue	Good-first-issue: docs, kueue-viz, SLSA[41,42,43,44,45,46]	Quotas, batch admission	Merged Go or build PR
Days 31–90	llm-d observability	llm_d_epp_* dashboards; spans, Collector recipes[76,77,78]	KV-cache usage, prefix-hit rate[75]	Recurring owner of an observability area
Days 31–90	llm-d-benchmark / vLLM	Benchmark runs and harness fixes[85,86,87,88,89]	Throughput vs goodput, load profiles	Published benchmark results or harness PR
Days 31–90	KServe	LLMInferenceService e2e/controller; #6260[41,42,43,44,45,46,83,84]	InferencePool, EPP, LWS roles	Merged controller or e2e PR
Days 91–180	llm-d-router / autoscaling	Scorer (#655, #2650) or autoscaler feature; proposal[27,33,72,74]	Prefix caching, prefill vs decode[74,83,91,109,115]	Merged feature; proposal reviewed
Days 91–180	vLLM	Read scheduler; metrics or tracing PRs[75,78,115,117,118]	Continuous batching, paged attention[115,117,118]	Engine-adjacent PR merged

Source: [27], [33], [41], [42], [43], [44], [45], [46], [72], [74], [75], [76], [77], [78], [83], [84], [85], [86], [87], [88], [89], [91], [109], [115], [117], [118]. Data as of 2026-09.

ORDER MATTERS MORE THAN SPEED

Every step up in ML knowledge comes with a project area that already needs it. That lets you ship something useful at each stage instead of studying first and contributing later[74,83,91,109,115].

08 SECTION

Limitations and data gaps

The data behind this ranking is partial. Monthly active contributors and merged PRs per month for October 2025 to September 2026 could not be obtained for any project. That is why the planned contributor chart was replaced with star and newcomer-issue snapshots. The only trend series available, OSS Insight's, is self-reported as degraded[54,101,102,103,104,105].

- LFX Insights scores cover only llm-d, vLLM and KServe. The window is stated only as 'past 365 days', and llm-d's stagnation flag conflicts with its release record[56,94,98,99,100].
- Help-wanted counts were collected only for vLLM, Kueue and KServe. Good-first-issue counts are missing for LWS, KAI Scheduler and SGLang[41,42,43,44,45,46,47,48,49].
- The Go share of llm-d-router itself was not measured. The 96.8% figure is for its predecessor repository[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21].
- Maintainer affiliations for GAIE, Kueue and vLLM are not published in their OWNERS or committers files[124,125,126].
- KServe's roadmap dates from 2024–2025, so its 2026 priorities are inferred from release notes[37,110].
- Employer demand rests on 3 postings from tier 3–4 aggregators, one already removed[69,70,71].

None of these gaps changes the ranking's basic logic, which rests on direct, tier 1 data: language shares, governance changes, SIG structure and dated issue counts[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,29,30,35,41,42,43,44,45,46]. They do mean that project activity should be judged again at the start of each stage of the roadmap rather than taken from this snapshot.

WHAT TO DO NEXT

What to do next

Put roughly two-thirds of your time into llm-d's Go repositories and SIGs, and use Kueue or LWS to get quick merged Go PRs while you learn the vocabulary. Since the 2026 hand-off, llm-d holds the routing, KV-cache and autoscaling logic that employers describe, and its simulator removes the need for a GPU[29,30,34,109]. Add KServe when you want controller work in a CNCF incubating project that a hiring posting names[22,23,24,25,26,69].

Treat vLLM as a learning resource first and a contribution target second. Its metrics and tracing are where an SRE's contributions fit naturally, and it is the project employers name most often in the sampled postings[69,70,71,75]. Avoid starting in GAIE: after v1.6.0 it covers APIs and conformance, and its meetings have moved to llm-d[29,30,31,32,33].

Watch three things over the next two quarters. First, whether llm-d's newcomer labels and SIG asks grow as it matures inside the CNCF. Second, whether KServe publishes a 2026 roadmap. Third, whether 2027 LFX and GSoC rounds include the core inference projects[37,62,63,64,65,110]. Any of these would lower the cost of getting started.

Numbered citations refer to 126 sources listed in the companion file **open-source-paths-into-llm-inference-sre-go-engineers.references.json**. Re-render with `--save-ref` to include them here.

APPENDIX A

Methodology and evidence quality

This report was produced by Boku, a multi-agent research pipeline. Research workstreams (primary, market, technical, case-study) gathered evidence from public sources; each claim was recorded with its sources and date. An independent fact-checking agent reviewed the corpus over 1 round(s) (final status: needs_revision) before synthesis and editing. Information dated within 365 days of publication is labelled current; older material is labelled historical.

TABLE 9

Evidence base

MEASURE	COUNT
Findings recorded	81
Findings used (after fact-check)	81
Findings rejected	0
Findings flagged as weakly supported	0
Distinct sources supporting usable findings	127
Tier 1 · Primary / official	107
Tier 2 · Authoritative secondary	8
Tier 3 · Specialist / analyst	10
Tier 4 · Community / unverified	2

Not verified

NOT VERIFIED BY FACT-CHECK

The fact checker could not verify the following, and statements in this report that touch on them should be treated as unconfirmed.

- No verified data available for AI chip market size in 2025
- No verified data available for NVIDIA AI chip revenue in 2025
- No verified data available for AMD AI chip market share in 2025
- No verified growth rate data for AI chip market in 2025

Limitations and unresolved questions

- Unverified: The AI chip market size in 2025 is projected to be $X billion
- Unverified: NVIDIA's AI chip revenue in 2025 will reach $X billion
- Unverified: AMD's market share in the AI chip market in 2025 is X%
- Unverified: The AI chip market is expected to grow at X% CAGR in 2025
- Monthly active contributors averaged over Oct 2025–Sep 2026 could not be obtained for any project, so the planned contributor bar chart is replaced by star and newcomer-issue snapshots.
- Merged pull requests per month were not collected for any project, so development velocity is shown only through release cadence.
- The only activity trend available is OSS Insight's cumulative PR-creator series. It self-reports as degraded, has missing months and jumps anomalously in September 2026, so it indicates direction only.
- 'Help wanted' counts were collected only for vLLM, Kueue and KServe, and good-first-issue counts are missing for LWS, KAI Scheduler, SGLang and several llm-d repositories.
- The Go share for llm-d-router itself was not measured. The 96.8% figure is for its predecessor repository, llm-d-inference-scheduler.
- LFX Insights health scores and organisation counts were found only for llm-d, vLLM and KServe, and the exact window they cover is not stated beyond 'past 365 days'.